

Day 108: Naive Bayes — Beginner Introduction

1. Day number

Day 108

2. Topic name

Probability-Based Classification with Naive Bayes

3. Connection to previous learning

You already learned basic **probability**.

Today, you will see how probability can help a machine-learning model answer questions such as:

- Is this email **Spam** or **Not Spam**?
- Is this review **Positive** or **Negative**?
- Does this customer belong to **Class A** or **Class B**?

The core idea is:

```
Observe some evidence
      ↓
Ask which class becomes more likely
      ↓
Predict the more likely class
```

Naive Bayes is therefore a **classification algorithm based on probability**.

4. Important topics

Class probability

A **class probability** tells us how likely a particular class is.

Suppose our classes are:

```
Spam
Not Spam
```

Before examining a new email, perhaps spam is less common:

```
Spam      → 30%
Not Spam  → 70%
```

These starting probabilities give the model some initial information about how common each class is.

Evidence idea

Evidence is information we observe about the new example.

For an email, evidence might include words such as:

```
"free"  
"winner"  
"meeting"  
"project"
```

Suppose the word "free" appears much more frequently in spam emails.

Seeing "free" gives us evidence that may increase the likelihood that the email belongs to the Spam class.

Think:

```
Before seeing evidence:  
Spam is possible.  
  
Evidence:  
The email contains "free".  
  
After considering evidence:  
Spam may become more likely.
```

Conditional probability intuition

Conditional probability asks something like:

| How likely is something when we already know something else?

For example:

| How likely is the word "free" to appear, given that an email is spam?

Conceptually:

```
P("free" | Spam)
```

Read that as:

| Probability of seeing "free" given that the email is Spam.

Naive Bayes uses ideas like this to connect **features/evidence** with **classes**.

You do not need to derive Bayes' theorem for this lesson. The important idea is that evidence can update which class appears more likely.

5. Foundational notes

Suppose we have training emails labeled:

Spam
Not Spam

The model examines which features tend to appear in each class.

For example:

Email	Contains "free"	Contains "meeting"	Class
1	Yes	No	Spam
2	Yes	No	Spam
3	Yes	Yes	Spam
4	No	Yes	Not Spam
5	No	Yes	Not Spam
6	No	No	Not Spam

From this tiny dataset, the model may notice:

"free" is common in Spam
"meeting" is common in Not Spam

Now suppose a new email contains:

"free"

Naive Bayes can use the training data to reason that the **Spam** class is more likely.

Importantly, the model does not simply use one fixed rule like:

if "free":
 Spam

Instead, it compares probabilities for the possible classes.

6. Why is it called “Naive” Bayes?

The word **naive** refers to a simplifying assumption.

Naive Bayes roughly assumes that the features are **independent of each other once we know the class**.

For example, imagine the email has these features:

```
contains "free"  
contains "winner"  
contains "offer"
```

Naive Bayes treats these pieces of evidence rather separately when combining them.

In real life, words are often related.

For example:

```
"free"  
"special offer"
```

may frequently appear together.

So the independence assumption is not always realistic.

That is why the method is called **naive**.

But this simple assumption makes the algorithm easy and efficient, and it can still work well for some classification tasks, especially text-related ones.

A beginner-friendly way to remember it:

Naive Bayes says: consider each clue in a simple, mostly separate way, then combine the clues using probability.

7. Easy example — Spam vs Not Spam

Suppose we have this tiny conceptual dataset:

Message	Contains "free"	Contains "meeting"	Class
A	Yes	No	Spam
B	Yes	No	Spam
C	Yes	Yes	Spam
D	No	Yes	Not Spam
E	No	Yes	Not Spam
F	No	No	Not Spam

Now a new message arrives:

```
"Free gift available today"
```

We simplify the message into features:

contains "free" = Yes
contains "meeting" = No

Now think about the training data.

Among Spam examples:

"free" appears frequently

Among Not Spam examples:

"free" appears rarely or not at all

So the evidence:

contains "free"

pushes our belief more strongly toward:

Spam

Therefore, the model may predict:

Spam

8. Problem statement

Use this tiny conceptual dataset:

Contains "free"	Contains "meeting"	Class
Yes	No	Spam
Yes	No	Spam
Yes	Yes	Spam
No	Yes	Not Spam
No	Yes	Not Spam
No	No	Not Spam

Now classify this new email:

contains "free" = Yes
contains "meeting" = No

Your goal is **not** to perform complicated probability calculations.

Instead, explain:

1. What are the two possible classes?
2. What evidence does the new email provide?
3. In which class is "free" more common?
4. In which class is absence of "meeting" more common?
5. Which class seems more likely after considering the evidence?

9. Concepts used

Classification

We are predicting a category:

Spam
or
Not Spam

Features

The evidence becomes model features.

For example:

contains_free
contains_meeting

Target / label

The target is:

class

with possible values:

Spam
Not Spam

Class probability

How likely is each class?

Conceptually:

Probability of Spam
Probability of Not Spam

Conditional probability

How common is some evidence when a particular class is known?

For example:

How common is "free" among Spam emails?

Evidence

Observed features of the new example.

Naive assumption

Features are treated as conditionally independent in the model's simplified probability calculation.

10. Thought process

Suppose a new email contains "free" but not "meeting".

Start with the possible classes:

Spam
Not Spam

Then inspect the evidence:

Evidence 1:
"free" is present

Evidence 2:
"meeting" is absent

Now compare the training examples.

Ask:

Does "free" appear more often in Spam or Not Spam?

From our tiny dataset:

Mostly Spam

Then ask:

Does this overall evidence resemble Spam examples
or Not Spam examples more strongly?

It resembles the Spam examples.

So conceptually:

```
Evidence
  ↓
"free" strongly associated with Spam
  ↓
Spam probability becomes relatively higher
  ↓
Predict Spam
```

That is the basic intuition behind Naive Bayes classification.

11. Beginner-friendly pseudocode

```
START

collect labeled training examples

identify possible classes:
    Spam
    Not Spam

learn how common each class is

for each feature:
    learn how common that feature is
    inside each class

receive a new email

observe its features

calculate a score/probability
for Spam using the evidence

calculate a score/probability
for Not Spam using the evidence

compare the two

if Spam is more likely:
    predict Spam
else:
    predict Not Spam

END
```

The machine-learning library handles the probability calculations for you.

12. Simple scikit-learn workflow

After understanding the concept manually, you can use scikit-learn.

One Naive Bayes model you may encounter for simple discrete/count-style features is:

```
from sklearn.naive_bayes import BernoulliNB
```

For features such as:

```
contains_free = 0 or 1  
contains_meeting = 0 or 1
```

the data could conceptually look like:

```
X:  
  
[1, 0]  
[1, 0]  
[1, 1]  
[0, 1]  
[0, 1]  
[0, 0]
```

And the labels:

```
y:  
  
Spam  
Spam  
Spam  
Not Spam  
Not Spam  
Not Spam
```

Then the general scikit-learn workflow is:

```
prepare X
prepare y

create Naive Bayes model

model.fit(X, y)

prepare new email features

model.predict(new_email)
```

Conceptually:

```
model = BernoulliNB()

model.fit(?, ?)

prediction = model.predict(?)
```

Your exercise is to determine what belongs in the missing parts rather than copying a complete solution.

13. Easy edge cases

A feature never appeared for one class

Imagine "winner" appears in some Spam emails but **never** in any Not Spam training email.

A direct probability calculation might otherwise create a zero probability.

Naive Bayes implementations commonly use a technique called **smoothing** to avoid probabilities becoming unusably zero.

For now, just remember:

┆ Libraries have a way to handle evidence that is rare or unseen.

You can study smoothing later.

Very tiny dataset

Suppose you have only:

```
2 Spam emails
2 Not Spam emails
```

The probabilities learned from such a tiny dataset may not represent real email behavior very well.

More representative training data usually gives more trustworthy patterns.

Weak evidence

Suppose the new email contains only very common words that occur equally often in both classes.

Then the evidence may not strongly favor either class.

The prediction may then depend more on the overall class frequencies and the other available evidence.

Unusual new example

If the new example is very different from everything in the training data, the model still produces a prediction, but you should interpret it carefully.

14. Expected classification explanation

For the exercise:

New email:

contains "free" = Yes
contains "meeting" = No

Look at the training examples.

The word:

"free"

appears strongly in the Spam examples.

The combination:

free = Yes
meeting = No

also resembles the Spam examples more closely.

Therefore, the expected conceptual result is:

Spam becomes more likely than Not Spam.

So the predicted class would likely be:

Spam

The important point is **why**:

| The observed evidence occurs more strongly with the Spam class in the training data.

15. Hint only

Think through the exercise like this:

```
Possible classes
↓
Spam / Not Spam
↓
Look at new evidence
↓
free = Yes
meeting = No
↓
Ask how common each clue is
inside each class
↓
Combine those clues
↓
Compare class likelihoods
↓
Choose more likely class
```

For the scikit-learn part, investigate this structure:

```
from sklearn.naive_bayes import BernoulliNB

model = BernoulliNB()

model.fit(...)
model.predict(...)
```

Do not focus on memorizing the Bayes theorem formula yet.

The main idea from **Day 108** is:

Naive Bayes uses probabilities learned from labeled examples to decide which class is most likely given the evidence it observes.